

ข้อจำกัดที่ควรรู้

- ผลทั้งหมดมาจากข้อมูลตัวแทน (ภาพคีย์บอร์ด QWERTZ จาก Kaggle) บนชุด Validation เท่านั้น
- ยังไม่ได้รันชุด Test และยังไม่ได้ทดสอบกับภาพถ่ายคีย์บอร์ด QWERTY จริง
- อักษรไทยบนปุ่มเป็นแบบสังเคราะห์ ยังพิสูจน์กับปุ่มไทยจริงไม่ได้
- ผู้ใช้ต้องแตะจุดอ้างอิง 4 จุดเอง
- รองรับเฉพาะ QWERTY แถวเฉียง ตัวอักษร A-Z และคีย์บอร์ดแบบ Ortholinear ที่แตะจุดถูกยังไม่ถูกปฏิเสธ

ก้าวต่อไป

- เก็บภาพคีย์บอร์ด QWERTY จริง 8-12 รุ่น (รวมแบบไทย-อังกฤษ)
- รันชุด Test ครั้งเดียว และรายงานผลแยกชี้ห้อยที่เคยเห็น/ไม่เคยเห็น
- ตรวจสอบุมคีย์บอร์ดอัตโนมัติ ลดการแตะ 4 จุด
- ทดสอบกับผู้ใช้จริงและ layout อื่น

เทคโนโลยีที่ใช้

- AI: Python · OpenCV · PyTorch · YOLO11n · ResNet18 · SciPy (Hungarian)
- Backend: FastAPI · MongoDB · Worker แยกโปรเซส
- Frontend: SvelteKit · TypeScript · Tailwind

รายงานฉบับเต็ม

รายละเอียดการออกแบบ การพัฒนา ผลการทดลอง และโค้ด อยู่ในไฟล์ docs/report ของโปรเจกต์

ผู้จัดทำ

[ชื่อ-สกุล] [รหัสนักศึกษา]
[ชื่อ-สกุล] [รหัสนักศึกษา]
อาจารย์ที่ปรึกษา: [ชื่ออาจารย์]
รายวิชา [รหัสวิชา] [ชื่อรายวิชา]
[คณะ / มหาวิทยาลัย]
ติดต่อ: [อีเมล]

MINI PROJECT

KeyCheck

ระบบตรวจคีย์แคปผิดตำแหน่ง จากภาพถ่ายใบเดียว ด้วย AI



ภาพบน: ต้นฉบับ ภาพล่าง: หลังสลับปุ่ม 2 ตัว (ภาพจำลองจากชุดข้อมูล)

ถ่ายภาพ → แตะ 4 จุด → รู้กันที่ว่าปุ่มไหนถูก ผิดหรือไม่แน่ใจ พร้อมแนะนำให้สลับปุ่มคูไหน

ปัญหาที่ต้องการแก้

- ถอดคีย์แคปไปทำความสะอาดแล้วใส่กลับผิดช่อง เพราะปุ่มตัวอักษรหน้าตาและขนาดเหมือนกัน
- ตรวจด้วยตาช้า และพลาดง่ายเมื่อผิดเพียงหนึ่งคู่จาก 26 ตัว
- ตัวอักษรบนปุ่มเป็นตัวเดียว ฟอนต์หลากหลาย มีไฟ RGB และบางรุ่นมีอักษรไทยร่วม

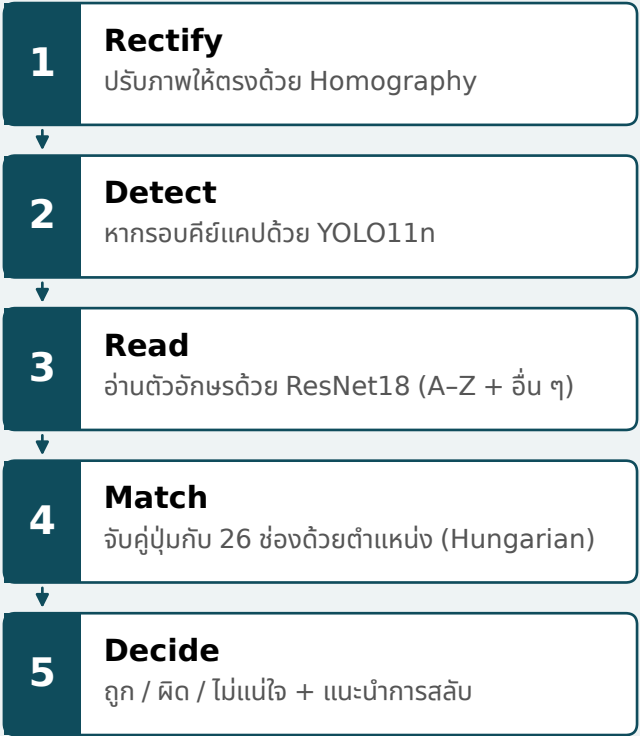
แนวคิด (Solution)

ถ่ายภาพคีย์บอร์ด 1 ภาพ แล้วแตะ 4 จุดที่ปุ่ม Q, P, M, Z ระบบจะบอกผลของทั้ง 26 ช่อง

อัลกอริทึมหลัก

- Homography (DLT 4 จุด):
ปรับภาพเอียงให้เป็นมุมมองตรงในหน่วยปุ่ม
- Hungarian algorithm $O(N^3)$:
จับคู่ปุ่ม-ช่องแบบหนึ่งต่อหนึ่งด้วย “ตำแหน่ง” พร้อม dummy และ gating จึงเห็นปุ่มที่ถูกสลับ
- กฎตัดสินหลายชั้น: ไม่แนใจ → จดตอบ, layout ไม่เข้ากัน → ปฏิเสธ
- กราฟหา swap pair / cycle: แนนำปุ่มที่ต้องสลับ
- Coordinate descent: จูน threshold จาก 124,416 ชุด เหลือประมาณ 100 ชุด
- ResNet18 (transfer learning) + YOLO11n:
อ่านตัวอักษรและตรวจจับปุ่ม

ระบบทำงานอย่างไร



ผลที่ผู้ใช้เห็น

- ผลรายช่องซ้อนบนภาพ: ✓ ถูก ! ผิด ? ไม่แนใจ
- คำแนะนำสลับปุ่ม (คู่หรือวงสามปุ่ม) เมื่อมันใจครบ
- ถ้าแตะจุดผิด/layout ไม่ตรง ระบบปฏิเสธและให้แตะใหม่

เว็บแอป: SvelteKit + FastAPI ประมวลผลผ่านคิวงาน
ตรวจไฟล์และสิทธิ์ผู้ใช้ก่อนเข้าโมเดล

ผลลัพธ์ที่ได้

0.854 vs 0.815

mAP50-95 ของตัวตรวจจับ YOLO11n เทียบ
Faster R-CNN (ชุด Validation)

0.37 vs ≈9.9 ซม.

เวลาฝึก YOLO11n เทียบ Faster R-CNN เร็วกว่าราว
27 เท่า (2.58 ล้านพารามิเตอร์)

สูงสุด 82%

OCR สำเร็จรูปอ่านปุ่มได้สูงสุดเพียง 82%
และปฏิเสธไม่อ่าน $\geq 16\%$
จึงเปลี่ยนเป็นโมเดลจำแนกเฉพาะทาง

1.311 vs 1.204

คะแนนรวมของ pipeline (เต็ม 1.35): YOLO11n
เทียบกับ baseline ที่ไม่ใช่ detector

ชุดข้อมูล: Kaggle QWERTZ 5,254 ภาพ
แบ่งชุดตามยี่ห้อคันข้อมูลรั้ว ผลทั้งหมดเป็นผลบน
Validation ของข้อมูลตัวแทน
ยังไม่ใช้ความแม่นยำของระบบกับภาพ QWERTY จริง